

Zadání třetího projektu SUI 2025/26

Šimon Strýček

25. listopadu 2025

Changelog:

- 25.11.: Explicitní zmínka o tom, že v konvoluční vrstvě není potřeba implementovat bias.
- 27.11.: Oprava data odevzdání aby korespondoval s ISem.
- 9.12.: Oprava formální definice tvaru vstupních tensorů.

Cílem třetího projektu v SUI je implementovat základní dva operátory pro tvorbu konvolučních sítí v prostředí minimalistické knihovny z loňského zadání druhého projektu. Prostředí pro vývoj a evaluaci algoritmů je k dispozici na Githubu¹. Termín odevzdání je středa 10. prosince 2025 ve 23:59:59, odevzdává se ve Studisu.

V rámci projektu máte vytvořit několik dílčích náležitostí:

1. Implementaci dopředného i zpětného průchodu pro konvoluční vrstvu (pro účely projektu postačí implementace bez biasu).
2. Implementaci dopředného i zpětného průchodu pro operaci max-pooling.

Rozhraní a náležitosti řešení Ústředním objektem v celém tomto projektu je tensor, instance třídy `Tensor`. Tento zapouzdřuje samotné hodnoty v tensoru (ve formě `numpy` pole) a navíc s sebou nese informaci o tom, jak tímto tensorem šířit chybu, a potažmo, jaký je gradient chyby podle tohoto tensoru. Hodnota tensoru a odpovídající gradient stejných rozměrů (`value` a `grad`) jsou považovány za jeho rozhraní a nelze měnit jejich název ani význam. Informace o zpětném šíření chyby (`back_op`) je realizována předáním `Callable`, která se volá při zpětném průchodu přímo v implementaci `Tensor`.

Pro implementaci dvou požadovaných operací (konvoluce a maxpool) je připraveno rozhraní v souboru `sui-solution.py`. Od implementovaných operací se očekává, že vytvoří nový tensor, naplní ho odpovídající hodnotou a dodají mu vhodnou `back_op`. Pro inspiraci je možné nahlédnout do vzorové implementace primitivních operátorů v souboru `primitives.py`.

Při implementaci můžete předpokládat, že:

- všechny tensoru vstupující do operací budou formálně mít batch size 1
- při budování neuronových sítí budou data považována za sloupcové vektory a při dopředném průchodu sítí tedy budou násobeny do matic zprava.

Pokud Vaše řešení zvládne i obecnější situace, ničemu to nevadí.

Implementovat budete v jazyce Python, odevzdávat budete jediný soubor `sui_torch.py`, ten musí dodat všechnu implementaci ve Vašem projektu. Při tvorbě Vašeho řešení nesmíte použít žádnou knihovnu pro automatickou diferenciaci nebo implementaci neuronových sítí. Naopak, je povinnost použít `numpy` pro implementaci jednotlivých operací.

Všechna místa pro implementaci jsou vyznačena pomocí `raise NotImplementedError()`, navíc můžete podle potřeby přidávat vlastní funkce či třídy. Do již přítomného kódu nezasahujte.

¹<https://github.com/StrycekSimon/sui-torch>, prosím klonovat, tak aby Vaše implementace zůstala soukromá.

Kontrola správnosti řešení Pro základní ověření správnosti je k dispozici základní sada jednotkových testů (`test.py`). Pokud k vývoji přistoupíte na základě studia (tj. nikoliv metodou pokus–omyl), lze předpokládat, že Vám tato sada může stačit. Dále je k dispozici jednoduchý demonstrační skript (`training.py`), který natrénuje vzorový model pro klasifikaci vstupního signálu do tří kategorií - sin/cos, náhodný šum nebo square wave. Jeho zdárný průchod bude součástí vyhodnocení projektů, je tedy nezbytné, aby s ním Vaše řešení bylo kompatibilní.

Doporučení pro řešení

- Pro zpětné šíření chyby potřebujete přinejmenším vědět, z čeho se nový tensor vypočítal.
- Pro ověření základní funkčnosti použijte přiložené testy.
- Trénovací skript dává smysl spouštět až po implementaci všech částí projektu.